TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A PROJECT REPORT

ON

UPTIME TRACKER: WEBSITE MONITORING AND AVAILABILITY TRACKING SYSTEM

Submitted By:

Abhishek Panthee	(THA080BCT002)
Alin Timelsana	(THA080BCT003)
Kiran Paudel	(THA080BCT019)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

December 2025

DECLARATION

We hereby declare that this project entitled **"UPTIME TRACKER: WEBSITE MONITORING AND AVAILABILITY TRACKING SYSTEM"** is our own original work. This project has been prepared and submitted as a DBMS academic project to the Department of Electronics and Computer Engineering, Thapathali Campus. The work presented in this report is genuine and has not been copied from any other source. This work has not been submitted elsewhere for academic evaluation or any other purpose.

Abhishek Panthee	(THA080BCT002)
Alin Timelsana	(THA080BCT003)
Kiran Paudel	(THA080BCT019)

Date: December 2025

COPYRIGHT

The author has agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the author has agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Thapathali Campus.

ACKNOWLEDGEMENT

We would like to thank Institute of Engineering, Tribhuvan University, for incorporating major project within the curriculum of Bachelor's in Electronics, Communication, and Information Engineering.

Special thanks to our supervisor, **Er. Rajad Shakya**, for his guidance, invaluable feedback, and constant support for the project. His expertise and encouragement have been pivotal in shaping this project. We are also grateful to our project coordinator, **Er. Umesh Kanta Ghimire**, for his continuous support and guidance.

We would like to express our appreciation to the Department of Electronics and Computer Engineering, Thapathali Campus, for their continuous assistance, recommendations during project selection. This has significantly helped in enabling our professional development.

We extend our sincere thanks to all the teachers, friends, and both direct and indirect contributors for their valuable insights, recommendations, and encouragement throughout this journey.

Abhishek Panthee	(THA080BCT002)
Alin Timelsana	(THA080BCT003)
Kiran Paudel	(THA080BCT019)

ABSTRACT

The Uptime Tracker is a web-based monitoring system designed to track website availability and performance metrics in real-time. In today's digital landscape, website downtime can significantly impact productivity, service delivery, and user trust. This project addresses the need for automated monitoring of college websites and web services by implementing a centralized tracking system similar to status.tcioe.edu.np and downdetector.com.

The system is built using Bun runtime with Next.js for the frontend and Elysia.js for the backend API, providing fast and efficient full-stack performance. PostgreSQL serves as the database for reliable data storage. The platform features user authentication, website registration and ownership management, automated ping checks at 5-minute intervals, and comprehensive analytics dashboards. It monitors HTTP/HTTPS availability, measures response times, and calculates hourly and daily average uptime statistics. The application supports both public and private monitoring modes, allowing website owners to manage visibility of their site's status.

The Uptime Tracker provides real-time status information, historical data visualization, and performance analytics that enable administrators to quickly identify and respond to downtime incidents. This transparency improves user experience and supports data-driven decisions for service reliability improvements. The project documentation is prepared using LaTeX for professional formatting.

Keywords—Website Monitoring, Uptime Tracking, Bun, Next.js, Elysia.js, PostgreSQL, Real-time Status, Performance Analytics, Web Application.

TABLE OF CONTENTS

DECLARATION . . . . . **i**

COPYRIGHT . . . . . **ii**

ACKNOWLEDGEMENT . . . . . **iii**

ABSTRACT . . . . . **iv**

LIST OF FIGURES . . . . . **vii**

LIST OF TABLES . . . . . **viii**

LIST OF ABBREVIATIONS . . . . . **viii**

LIST OF SYMBOLS . . . . . **ix**

1. Introduction **1**
- 1.1. Background 1
- 1.2. Problem Statement 1
- 1.3. Objectives 1
- 1.4. Scope and Limitations 2

2. LITERATURE REVIEW **3**
- 2.1. Introduction to Website Monitoring 3
- 2.2. Importance of Uptime Monitoring 3
- 2.3. Existing Monitoring Solutions 3
- 2.4. Technical Approaches to Monitoring 4
- 2.5. Database and Technology Considerations 4
- 2.6. Identified Research Gaps 4

3. REQUIREMENTS ANALYSIS **6**
- 3.1. Introduction 6
- 3.2. Functional Requirements 6
- 3.3. Non-Functional Requirements 7
- 3.4. System Requirements 8
- 3.5. Use Case: Add Website 8
- 3.6. Use Case: View Status 8

4. SYSTEM ARCHITECTURE AND METHODOLOGY **9**
- 4.1. Introduction 9

4.2. System Architecture Overview 9
4.3. Technology Stack 9
4.4. Database Design 10
4.5. API Design 13
4.6. Monitoring Service 13
4.7. User Interface Design 13
4.8. Development Methodology 13
4.9. Deployment 14

5. EXPECTED OUTCOMES 15
5.1. System Deliverables 15
5.2. Expected Performance 15
5.3. Documentation 15
5.4. Success Criteria 16

6. CONCLUSION 17
6.1. Summary 17
6.2. Expected Contributions 17
6.3. Feasibility 17
6.4. Conclusion 17

APPENDICES 18

A. PROJECT BUDGET 18
A.1. Software and Development Tools 18
A.2. Hosting and Cloud Services 18
A.3. Total Budget Summary 19

B. PROJECT TIMELINE 20
B.1. Gantt Chart 20
B.2. Milestone Summary 20

REFERENCES 21

LIST OF FIGURES

4.1. High-Level System Architecture 9
4.2. Database Entity Relationship Diagram 10
B.1. Project Gantt Chart 20

LIST OF TABLES

2.1. Comparison of Website Monitoring Solutions 4
3.1. Hardware and Software Requirements 8
4.1. Technology Stack Overview 10
4.2. Users Table Schema 11
4.3. Ownership Table Schema 11
4.4. Analytics Table Schema 11
4.5. Average Hour Table Schema 11
4.6. Average Day Table Schema 12
4.7. Refresh Tokens Table Schema 12
4.8. Password Reset Tokens Table Schema 12
4.9. Planned API Endpoints 13
5.1. Expected Metrics 15
A.1. Development Tools (No Cost) 18
A.2. Hosting and Cloud Services Cost 18
A.3. Total Estimated Budget 19
B.1. Project Milestones 20

LIST OF ABBREVIATIONS

API	Application Programming Interface
DBMS	Database Management System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
REST	Representational State Transfer
SLA	Service Level Agreement
SQL	Structured Query Language
SSR	Server-Side Rendering
UI	User Interface

1. INTRODUCTION

1.1. Background

Website availability is critical for organizations, with downtime costing businesses an average of $5,600 per minute. Educational institutions like Thapathali Campus depend on web-based systems for student information, examination portals, and learning platforms. When these services experience downtime, users lack visibility into service status.

Commercial monitoring solutions require recurring costs, while open-source alternatives need technical expertise. The Uptime Tracker project aims to create an accessible web application for automated website monitoring using Hypertext Transfer Protocol (HTTP)/Hypertext Transfer Protocol Secure (HTTPS) protocols, historical analysis with Structured Query Language (SQL) database, and public status pages built with modern Application Programming Interface (API) architecture.

1.2. Problem Statement

Many institutions lack centralized systems to monitor web service status. Key challenges include:

- **Lack of proactive monitoring:** Downtime goes undetected until users report issues
- **No historical visibility:** Administrators lack data on uptime patterns and trends
- **User uncertainty:** Users cannot determine if issues are service-related or local
- **Decentralized information:** No single authoritative status source exists

1.3. Objectives

1.3.1. Primary Objectives

1. Design a web application for automated uptime monitoring using Representational State Transfer (REST) API
2. Develop backend service performing HTTP/HTTPS checks at 5-minute intervals
3. Create Database Management System (DBMS) schema using PostgreSQL for analytics data
4. Build interactive dashboard with real-time status and User Interface (UI) visualizations
5. Implement user authentication with JSON Web Token (JWT) tokens

1.3.2. Secondary Objectives

1. Provide public status pages for registered websites
2. Enable privacy controls for monitoring data visibility
3. Generate aggregate statistics and trend analysis

1.4. Scope and Limitations

1.4.1. Scope

The system includes: user management with authentication; website registration interface; automated HTTP/HTTPS monitoring every 5 minutes; PostgreSQL database for ping results and aggregates; analytics dashboard with visualizations; and public status pages with privacy controls.

1.4.2. Limitations

- Limited to HTTP/HTTPS protocols only
- 5-minute check intervals (balance of speed vs. resources)
- Single-location monitoring
- No email/SMS alerting in initial version

2. LITERATURE REVIEW

2.1. Introduction to Website Monitoring

Website monitoring involves automated systems that periodically test whether websites are accessible using HTTP/HTTPS protocols. Modern monitoring tracks availability, response time, and HTTP status codes to help administrators measure service quality.

2.2. Importance of Uptime Monitoring

Website downtime carries significant costs, research indicates IT downtime costs organizations an average of $5,600 per minute. For educational institutions, downtime disrupts learning activities and access to critical resources. Proactive monitoring enables early detection, supporting Service Level Agreement (SLA) compliance and capacity planning.

2.3. Existing Monitoring Solutions

2.3.1. Commercial Platforms

Pingdom offers comprehensive multi-location monitoring with synthetic transactions and analytics, though pricing scales with features. **UptimeRobot** provides a freemium model with 5-minute checks on the free tier. **StatusCake** and **Datadog** offer additional options for different organizational needs.

2.3.2. Open-Source Solutions

Uptime Kuma is a popular self-hosted solution with a clean interface supporting HTTP, TCP, and DNS monitoring. **Nagios** and **Zabbix** offer enterprise-grade capabilities but require significant configuration expertise. **Prometheus with Blackbox Exporter** enables monitoring in cloud-native environments.

2.3.3. Comparison of Solutions

Table 2.1 summarizes key characteristics of existing monitoring solutions:

Table 2.1. *Comparison of Website Monitoring Solutions*

Solution	Type	Min Interval	Status Page	Self-Hosted
Pingdom	Commercial	1 minute	Yes	No
UptimeRobot	Freemium	5 minutes	Yes	No
StatusCake	Freemium	5 minutes	Yes	No
Uptime Kuma	Open-Source	20 seconds	Yes	Yes
Nagios	Open-Source	1 minute	Plugin	Yes
Zabbix	Open-Source	1 minute	Built-in	Yes

2.4. Technical Approaches to Monitoring

2.4.1. HTTP-Based Monitoring

HTTP monitoring sends requests to URLs and analyzes responses. Status codes indicate outcomes: 2xx codes indicate success, 4xx client errors, and 5xx server errors. Response time measurement captures total time from request to completion, providing performance insight beyond availability.

2.4.2. Check Intervals and Data Aggregation

A 5-minute interval balances detection speed with resource consumption, generating 288 data points per day per site. **Hourly aggregates** summarize checks into average response time and availability percentage. **Daily aggregates** enable long-term trend analysis while managing storage efficiently.

2.5. Database and Technology Considerations

Monitoring systems generate time-series data. PostgreSQL handles moderate DBMS workloads effectively with temporal data types and SQL aggregation capabilities.

Modern JavaScript/TypeScript ecosystems support full-stack development:

- **Next.js** provides React-based Server-Side Rendering (SSR) applications
- **Bun** offers faster runtime
- **Elysia.js** provides type-safe REST API development

2.6. Identified Research Gaps

Existing solutions present gaps:

- Commercial solutions require costly subscriptions
- Enterprise tools have steep learning curves
- Self-hosted options require specialized skills
- Generic solutions lack customization flexibility

The Uptime Tracker addresses these gaps with a straightforward, modern web-based monitoring solution accessible to users with standard development experience.

3. REQUIREMENTS ANALYSIS

3.1. Introduction

This chapter documents functional requirements specifying system capabilities, non-functional requirements defining quality attributes, and technical specifications.

3.2. Functional Requirements

3.2.1. User Management

- **FR-U1:** User registration with email/password validation
- **FR-U2:** Authentication via email/password with session tokens
- **FR-U3:** Session management with logout capability
- **FR-U4:** Secure password storage using hashing

3.2.2. Website Management

- **FR-W1:** Add websites for monitoring with URL validation
- **FR-W2:** Ownership association with users
- **FR-W3:** Public/private visibility control
- **FR-W4:** Edit and delete website registrations

3.2.3. Monitoring Functionality

- **FR-M1:** Automated 5-minute interval checks
- **FR-M2:** HTTP/HTTPS protocol support
- **FR-M3:** Response time measurement in milliseconds
- **FR-M4:** Status determination from HTTP codes
- **FR-M5:** Data recording with timestamps

3.2.4. Data Aggregation

- **FR-A1:** Hourly statistics computation
- **FR-A2:** Daily aggregation for trend analysis
- **FR-A3:** Historical data retention

3.2.5. User Interface

- **FR-I1:** Dashboard with status overview
- **FR-I2:** Visual status indicators (colors/icons)
- **FR-I3:** Response time graphs and trends
- **FR-I4:** Uptime statistics display
- **FR-I5:** Public status page for public websites

3.3. Non-Functional Requirements

3.3.1. Performance

- **NFR-P1:** Response time < 500ms under normal load
- **NFR-P2:** Support 100+ simultaneous website monitors
- **NFR-P3:** Check timing accuracy within 30 seconds

3.3.2. Reliability

- **NFR-R1:** 99% monitoring service uptime
- **NFR-R2:** Data integrity without loss or corruption
- **NFR-R3:** Automatic failure recovery

3.3.3. Security

- **NFR-S1:** Passwords hashed using bcrypt
- **NFR-S2:** Secure session tokens over HTTPS
- **NFR-S3:** Authorization checks on protected resources
- **NFR-S4:** Input validation to prevent injection

3.3.4. Usability

- **NFR-U1:** Responsive design for all devices
- **NFR-U2:** Intuitive navigation with minimal steps
- **NFR-U3:** Clear feedback for user actions

3.4. System Requirements

Table 3.1. *Hardware and Software Requirements*

Component	Minimum	Recommended
Processor	Dual-core 64-bit	Quad-core 64-bit
Memory	4 GB RAM	8 GB RAM
Storage	10 GB	50 GB SSD
OS	Windows 10+, macOS 12+, Linux	—
Runtime	Bun 1.0+	Latest stable
Database	PostgreSQL 14+	PostgreSQL 16

3.5. Use Case: Add Website

- **Actor:** Authenticated User
- **Flow:** User enters URL → System validates → Creates ownership record → Monitoring begins
- **Postcondition:** Website registered and monitored

3.6. Use Case: View Status

- **Actor:** Any User (public) / Authenticated User (private)
- **Flow:** Navigate to status page → System retrieves data → Displays status, response times, uptime

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. Introduction

This chapter presents the architectural design of the Uptime Tracker system, detailing the component structure, DBMS schema, and API design. The architecture follows a three-tier pattern separating presentation, business logic, and data storage.

4.2. System Architecture Overview

The Uptime Tracker employs a modern web architecture with four components:

1. **Frontend:** Next.js-based UI with dashboards and status pages
2. **Backend API:** Elysia.js server with REST endpoints
3. **Database:** PostgreSQL storing users and monitoring data
4. **Monitoring Service:** Background process executing HTTP checks

Figure 4.1 illustrates the system architecture.

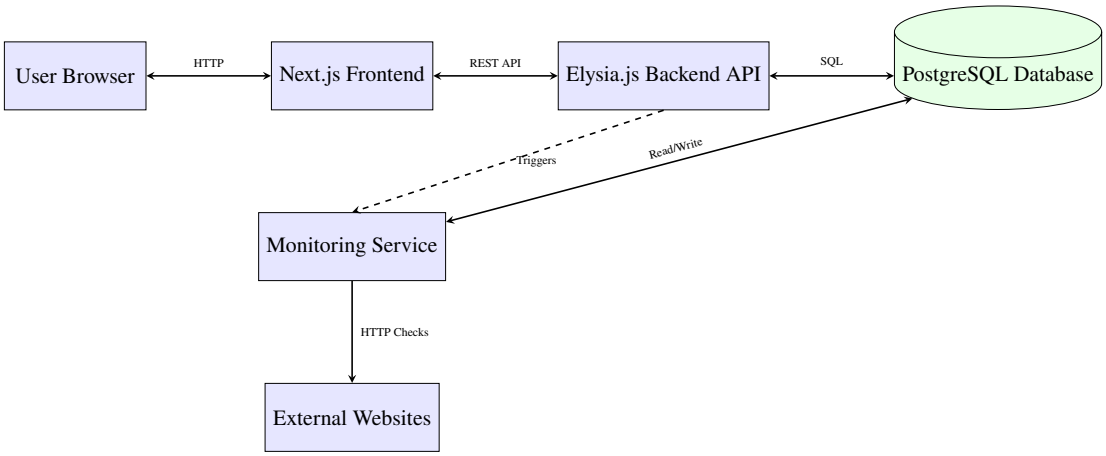

Figure 4.1. *High-Level System Architecture*

4.3. Technology Stack

The technology stack for this project has been selected based on modern best practices, performance requirements, and development efficiency. Table 4.1 summarizes the key technologies:

Key technology features: **Bun** provides native TypeScript support with faster performance; **Next.js** offers server-side rendering and file-based routing; **Elysia.js** provides type-safe API development; **PostgreSQL** ensures ACID compliance with efficient temporal queries.

Table 4.1. *Technology Stack Overview*

Component	Technology	Purpose
Runtime	Bun	JavaScript/TypeScript execution
Frontend	Next.js	User interface and client-side logic
Backend	Elysia.js	REST API and business logic
Database	PostgreSQL	Data persistence and queries
Language	TypeScript	Type-safe development
Styling	Tailwind CSS	UI styling and responsiveness

4.4. Database Design

4.4.1. Entity Relationship Model

The database schema consists of seven primary tables managing users, authentication tokens, website ownership, and monitoring data. Figure 4.2 shows the entity relationships.

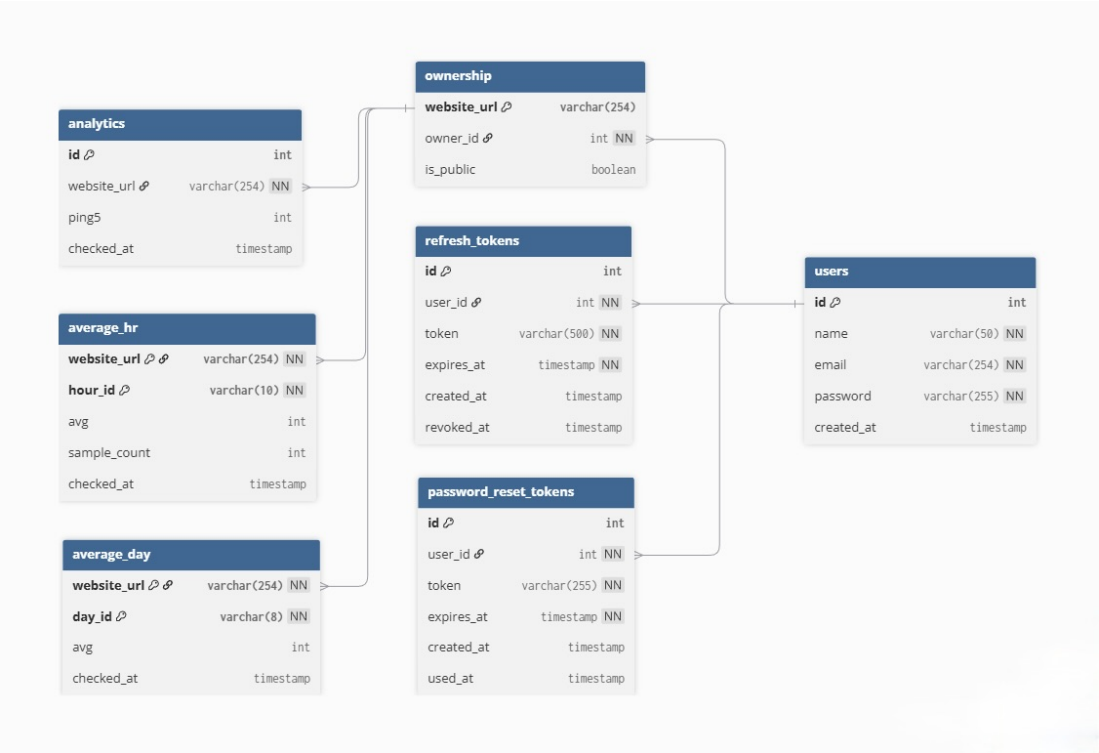

Figure 4.2. *Database Entity Relationship Diagram*

4.4.2. Table Specifications

Users Table

The users table stores account credentials and profile information:

Table 4.2. *Users Table Schema*

Column	Type	Constraints	Description
id	INT	PRIMARY KEY, NOT NULL	Auto-incrementing identifier
name	VARCHAR(38)	NOT NULL	User display name
email	VARCHAR(254)	NOT NULL	User email address
password	VARCHAR(255)	NOT NULL	Hashed user password
created_at	TIMESTAMP	NOT NULL	Account creation time

Ownership Table

The ownership table links users to their monitored websites:

Table 4.3. *Ownership Table Schema*

Column	Type	Constraints	Description
website_url	VARCHAR(254)	PRIMARY KEY	Website URL to monitor
owner_id	INT	NOT NULL	Reference to users.id
is_public	BOOLEAN	NOT NULL	Public visibility flag

Analytics Table

The analytics table records individual ping results:

Table 4.4. *Analytics Table Schema*

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Auto-incrementing identifier
website_url	VARCHAR(254)	FOREIGN KEY, NOT NULL	Reference to ownership
ping5	INTEGER	NULL	Response time in milliseconds
checked_at	TIMESTAMP	DEFAULT NOW()	Check timestamp

Average Hour Table

Stores hourly aggregated statistics:

Table 4.5. *Average Hour Table Schema*

Column	Type	Constraints	Description
website_url	VARCHAR(254)	PRIMARY KEY, NOT NULL	Reference to ownership
hour_id	VARCHAR(10)	PRIMARY KEY, NOT NULL	Hour identifier
avg	INT	NOT NULL	Average response time
sample_count	INT	NOT NULL	Number of samples
checked_at	TIMESTAMP	NOT NULL	Aggregation timestamp

Average Day Table

Stores daily aggregated statistics:

Table 4.6. *Average Day Table Schema*

Column	Type	Constraints	Description
website_url	VARCHAR(254)	PRIMARY KEY, NOT NULL	Reference to ownership
day_id	VARCHAR(8)	PRIMARY KEY, NOT NULL	Day identifier
avg	INT	NOT NULL	Daily average response time
sample_count	INT	NOT NULL	Number of samples
checked_at	TIMESTAMP	NOT NULL	Aggregation timestamp

Refresh Tokens Table

Stores JWT refresh tokens for user authentication sessions:

Table 4.7. *Refresh Tokens Table Schema*

Column	Type	Constraints	Description
id	INT	PRIMARY KEY	Auto-incrementing identifier
user_id	INT	NOT NULL	Reference to users.id
token	VARCHAR(380)	NOT NULL	JWT refresh token
expires_at	TIMESTAMP	NOT NULL	Token expiration time
created_at	TIMESTAMP	NOT NULL	Token creation time
revoked_at	TIMESTAMP	NULL	Token revocation time

Password Reset Tokens Table

Stores tokens for password reset functionality:

Table 4.8. *Password Reset Tokens Table Schema*

Column	Type	Constraints	Description
id	INT	PRIMARY KEY	Auto-incrementing identifier
user_id	INT	NOT NULL	Reference to users.id
token	VARCHAR(255)	NOT NULL	Password reset token
expires_at	TIMESTAMP	NOT NULL	Token expiration time
created_at	TIMESTAMP	NOT NULL	Token creation time
used_at	TIMESTAMP	NULL	Token usage timestamp

4.5. API Design

The backend will expose RESTful API endpoints organized by resource type. Table 4.9 summarizes the planned endpoints:

Table 4.9. *Planned API Endpoints*

Method	Endpoint	Description
POST	/api/auth/register	Create new user account
POST	/api/auth/login	Authenticate and receive token
POST	/api/auth/logout	Invalidate session
GET	/api/websites	List user's monitored websites
POST	/api/websites	Add new website to monitor
PUT	/api/websites/:id	Update website settings
DELETE	/api/websites/:id	Remove website from monitoring
GET	/api/status/:url	Get current status of website
GET	/api/analytics/:url	Get historical data for website
GET	/api/public/status	Get all public website statuses

4.6. Monitoring Service

The monitoring service executes the following steps:

- Scheduler triggers checks every 5 minutes
- Retrieves URLs from ownership table
- Sends parallel HTTP requests
- Records response times in analytics table
- Updates hourly aggregates

4.7. User Interface Design

The frontend uses a component-based architecture with: Landing Page, Dashboard (website status grid), Website Detail (historical charts), Add Website form, Public Status Page, and Settings. The dashboard displays website cards with URL, status indicator, response time, and uptime percentage.

4.8. Development Methodology

The project follows iterative development with incremental features, regular integration, and prioritized backlog. Development phases:

- Foundation—setup and authentication
- Core Features—monitoring service
- Interface—dashboard and visualizations
- Polish—testing and error handling

4.9. Deployment

The system supports single-server deployment with all components, or separate services for scaling. Requirements: Linux server with Bun, PostgreSQL access, and HTTPS configuration.

5. EXPECTED OUTCOMES

5.1. System Deliverables

Upon completion, the project will deliver:

5.1.1. Web Application

- User registration, authentication, and profile management
- Website monitoring interface with real-time status display
- Response time visualization and historical statistics
- Public status page for opted-in websites

5.1.2. Backend Services

- RESTful API for all application functionality
- Monitoring service with automated 5-minute checks
- Data aggregation for hourly and daily statistics

5.1.3. Database

PostgreSQL database storing user accounts, website registrations, ping results, and computed aggregates.

5.2. Expected Performance

Table 5.1. *Expected Metrics*

Metric	Expected Value
Check Interval Accuracy	± 30 seconds
Downtime Detection	Within 5-10 minutes
API Response Time	< 200ms
Concurrent Users	Up to 100
Website Capacity	Up to 500 monitors

5.3. Documentation

The project will produce: database schema documentation, API reference, deployment guide, user manual, and final project report.

5.4. Success Criteria

- All primary objectives implemented and operational
- Automated checks execute without manual intervention
- Dashboard displays accurate real-time status
- Public status page accessible without authentication
- Complete technical documentation

6. CONCLUSION

6.1. Summary

This proposal presents the Uptime Tracker, a web-based monitoring system for tracking website availability and response times. The system uses Next.js frontend, Elysia.js backend, and PostgreSQL database on Bun runtime.

6.2. Expected Contributions

The project will deliver: a functional web application with 5-minute monitoring intervals, PostgreSQL database for analytics, interactive dashboard with status visualization, public status pages, and complete documentation.

6.3. Feasibility

- **Technical:** All technologies are well-documented and actively maintained
- **Time:** 3-month timeline with buffer for challenges
- **Resource:** Open-source tools with free-tier cloud options
- **Scope:** Realistic for a DBMS academic project

6.4. Conclusion

The Uptime Tracker represents a practical application of database management combined with modern web development. The project team is confident in successful completion within the proposed timeline, delivering a functional website monitoring system for Thapathali Campus.

A. PROJECT BUDGET

The total budget required for the successful completion of the Uptime Tracker project is estimated to be between NPR 5,000 and NPR 10,000. Since this is a software-based project, the costs are primarily associated with cloud services, domain registration, and miscellaneous documentation expenses.

A.1. Software and Development Tools

All development tools used in this project are open-source and free:

Table A.1. *Development Tools (No Cost)*

Tool	Description	Cost
Bun Runtime	JavaScript/TypeScript runtime	Free
Next.js	React framework for frontend	Free
Elysia.js	Backend web framework	Free
PostgreSQL	Open-source database	Free
Visual Studio Code	Code editor	Free
Git	Version control system	Free
Tailwind CSS	CSS framework	Free
	Subtotal	**Free**

A.2. Hosting and Cloud Services

For deployment and hosting, the following services may be required:

Table A.2. *Hosting and Cloud Services Cost*

Service	Description	Estimated Cost
Domain Registration	.com or .np domain (1 year)	NPR 1,500 - 2,500
VPS Hosting	Basic cloud server (3 months)	NPR 2,000 - 4,000
PostgreSQL Hosting	Managed database (optional)	Free tier available
SSL Certificate	Let's Encrypt	Free
	Subtotal	**NPR 3,500 - 6,500**

Note: Free-tier options from Vercel (frontend), Railway (backend), and Supabase (database) can reduce hosting costs to zero during development and initial deployment.

A.3. Total Budget Summary

Table A.3. *Total Estimated Budget*

Category	**Estimated Cost**
Development Tools	Free
Hosting and Cloud Services	NPR 3,500 - 6,500
Total Estimated Cost	**NPR 3,500 - 6,500**

Note: The actual costs may vary depending on the hosting provider chosen and the duration of service usage. Using free-tier cloud services can significantly reduce the overall budget.

B. PROJECT TIMELINE

The project will be executed over a period of approximately 3 months, divided into several key phases. Each phase includes specific tasks and milestones to ensure steady progress toward project completion.

B.1. Gantt Chart

The Gantt chart in Figure B.1 provides a visual representation of the project timeline, showing task durations, dependencies, and key milestones.

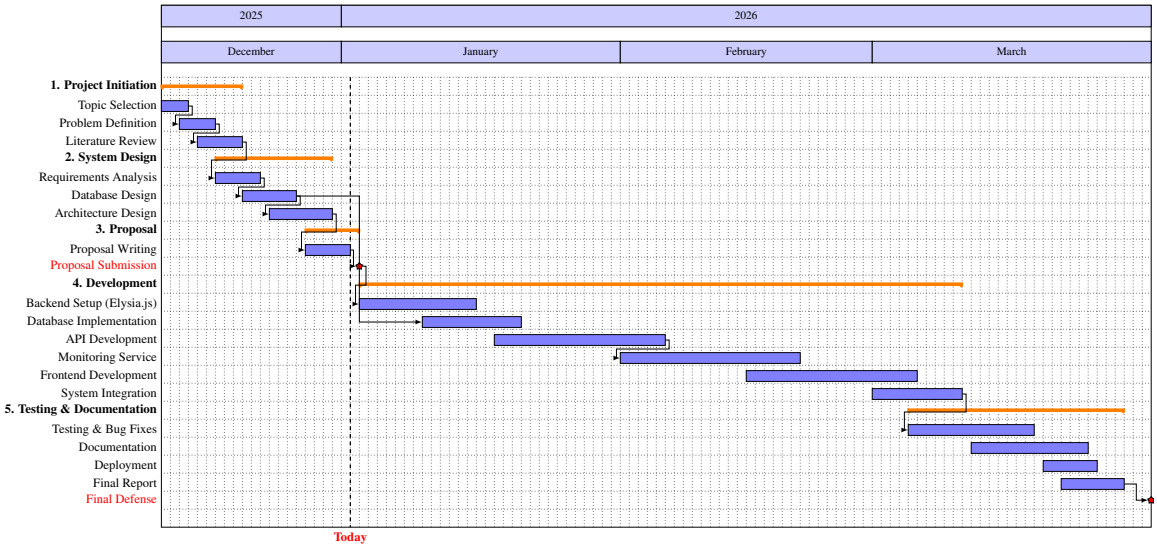

Figure B.1. *Project Gantt Chart*

B.2. Milestone Summary

Key project milestones are summarized in Table B.1:

Table B.1. *Project Milestones*

Milestone	Target Date	Deliverable
Project Start	December 12, 2025	Topic selection and planning
Design Complete	December 30, 2025	Database schema and architecture
Proposal Submission	January 2, 2026	Complete project proposal
Backend Complete	February 15, 2026	Working API and monitoring service
Development Complete	March 10, 2026	Fully integrated application
Final Report	March 28, 2026	Complete project documentation
Final Defense	March 31, 2026	Project presentation and demo

REFERENCES

[1] Ponemon Institute, "Cost of data center outages," *Ponemon Institute Research Report*, 2016, sponsored by Emerson Network Power.

[2] L. Lam, "Uptime kuma - a self-hosted monitoring tool," https://github.com/louislam/uptime-kuma, 2023, accessed: 2025-12-15.

[3] Vercel, "Next.js - the react framework for production," https://nextjs.org/, 2024, accessed: 2025-12-15.

[4] SaltyAom, "Elysia - ergonomic framework for humans," https://elysiajs.com/, 2024, accessed: 2025-12-15.

[5] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*. The PostgreSQL Global Development Group, 2024, available at https://www.postgresql.org/docs/16/.

[6] R. T. Fielding, "Architectural styles and the design of network-based software architectures," *Doctoral Dissertation, University of California, Irvine*, 2000.

[7] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2016.

[8] Microsoft, "Typescript - javascript with syntax for types," https://www.typescriptlang.org/, 2024, accessed: 2025-12-15.

[9] T. Berners-Lee, R. Fielding, and H. Frystyk, "Hypertext transfer protocol – http/1.0," *RFC 1945, IETF*, 1996.